

Smart Semantic Resume Search Engine

An intelligent recruitment engine powered by Google Gemini embeddings and vector indexes. This architecture completely abandons traditional keyword match strategies in favor of mathematical contextual awareness.

Project Architecture

1. **Extraction & Standardization**: Text profiles are curated under `resumes/`` spanning freshers, domain specialists, and non-tech profiles.
2. **Dense Vector Mapping**: Text blocks are converted into high-dimensional geometric vectors using the uniform `text-embedding-004`` model.
3. **Double-Engine Querying**: Incoming queries are processed in parallel using a manual **NumPy** vector space mathematical array and an accelerated **FAISS** index.
4. **LLM Explainer**: The target match profile is parsed alongside the job requirements by `gemini-2.5-flash`` to generate instant context strings for recruitment interfaces.

The Semantic Advantage: Where Keyword Searches Fail

Example Scenario

Job Description: "Looking for a Cloud Infrastructure expert who can deploy Kubernetes clusters and configure infrastructure as code using Terraform."

Winning Profile (`resume_10.txt``): "Hannah Abbott. DevOps Engineer. Expert in Docker, Kubernetes, CI/CD pipelines (Jenkins, GitHub Actions), and Terraform Infrastructure as Code."

Why Keywords Fail Here

If a recruiter used a traditional keyword string search looking for **"Cloud Infrastructure Expert"**, Hannah Abbott's profile **would fail to return or rank terribly low**.

Her profile never explicitly uses the phrase **"Cloud Infrastructure Expert"**.

Instead, she lists the exact ecosystem components (**Docker**, **Kubernetes**, **Terraform**).

Why Vector Search Succeeds

The Gemini embedding model understands that `Kubernetes` and `Terraform` are mathematically and conceptually adjacent to `Cloud Infrastructure`. It assigns them a shared coordinate space.

When calculating the vector distance, the system registers a high similarity score of **~0.68**, easily crossing our strict **0.55 validation threshold** and instantly surfacing her as the top candidate.

Numpy vs. FAISS Benchmarking Analysis

Parameter	NumPy Array Searching	FAISS (Facebook AI Similarity Search)
Execution Speed	Moderate (~0.0005s for small datasets)	Extremely Low Latency (~0.0002s)
Setup Cost	Zero overhead (Native Matrix multiplication)	Requires building, training, and allocating a dedicated index map
Scaling Capability	Exponentially degrades ($O(N^2)$ checking overhead)	Sublinear scaling ($O(\log N)$ via quantization structural clustering)

Performance Architectural Summary

For smaller candidate pools (under 5,000 files), writing standard matrix algorithms natively in NumPy is perfectly adequate and requires zero index maintenance. However, as datasets grow to enterprise scales (millions of applications), **FAISS** is essential. It transforms brute-force spatial calculation loops into near-instantaneous indexed vector lookups.